CTRE - Team Walkthrough (Detailed)

A complete guide for explaining the Cyber Threat Reconstruction Engine (CTRE) as a group, written for people who are new to cybersecurity. It starts with a plain-language crash course that everyone should read together, then gives each presenter a detailed part: what the idea is, why it matters to the project, what the code actually does and in which files, a script to say out loud, an analogy, and the questions you are likely to get.

Presenters and their parts:

- **Vivek**: the problem and the big picture (Part 1)
- **Kshitij**: disk forensics - reading the file system and catching fake timestamps (Part 2)
- **Moksh**: network evidence - reading traffic without decrypting it, and connecting it to the disk (Part 3)
- **Aranck**: the app, testing, safety, and results (Part 4)

Suggested order: everyone presents the crash course idea they need, Vivek sets up the problem and the architecture, Kshitij and Moksh explain the two halves of the analysis, and Aranck shows it running and reports what it found.

Part 0: Crash course (read this first, together)

You do not need a security background to present this project. You need about ten ideas. Here they are in plain language.

What is digital forensics and incident response?

When a computer is attacked or misused, investigators try to work out what happened: what got in, what it did, and whether anything was taken. This is "digital forensics." Doing it during or just after an attack is "incident response," often shortened to DFIR. The investigator does not work on the live, running machine, because that could destroy evidence. Instead they take copies of the important data and study the copies. CTRE is a tool that studies those copies.

The three kinds of evidence we use

An attack leaves marks in more than one place:

- **The file system** (the disk): new files appear, old ones change.
- **The file system's own logbook** (a journal): the operating system writes down each change as it happens.
- **The network capture**: a recording of the traffic that went in and out of the machine.

Normally each of these is opened in a different tool, and a human tries to line them up by hand. That is slow and easy to get wrong. CTRE reads all three together.

What is NTFS, a file, and the MFT?

NTFS is the file system Windows uses. A file system is the filing cabinet that keeps track of every file: its name, where its data lives, and when it was created or changed. In NTFS, the master index of that filing cabinet is called the **MFT** (Master File Table). There is one entry in the MFT for every file, and that entry holds the file's timestamps.

The two sets of timestamps (this is the key idea)

Here is the single most important fact for this project. Windows stores a file's timestamps in two different places inside its MFT entry:

- **SI** (short for `$STANDARD_INFORMATION`): the times you see when you look at a file's properties. These

  are **easy to change** with ordinary tools.
- **FN** (short for `$FILE_NAME`): a second copy of the times, kept by the Windows kernel for its own use. These are **hard to change** from normal programs.

So a file carries an "easy to fake" clock and a "hard to fake" clock. If the two disagree, something altered the easy one. That disagreement is what we hunt for.

The change journal (the tamper-proof logbook)

NTFS also keeps a running logbook called the **change journal** (`$UsnJrnl`). Every time a file is created, renamed, written to, or deleted, the system appends a line to this logbook with the time and what happened. It is append-only, so it is a second, independent record of when things really happened. If a file's "easy" clock says it was created in 2019 but the logbook only first mentions the file last week, the 2019 date is a lie.

What is a packet capture (pcap)?

When computers talk over a network, they send small chunks of data called packets. A **packet capture** (a ".pcap" file) is a saved recording of those packets. From it you can see which machines talked, on which network ports, how much data moved, and when. Tools like Wireshark open these files; our project reads them directly.

Encryption, and why we do NOT break it

Most web traffic today is **encrypted**: scrambled so that only the two ends can read it. A packet capture of encrypted traffic shows you the envelope but not the letter inside: you can see that machine A sent 2 megabytes to machine B at 3pm, but not what those 2 megabytes contained. CTRE deliberately does **not** try to unscramble (decrypt) this. A tool that reads the secret keys out of a computer's memory and unscrambles traffic is a general-purpose eavesdropping tool, and that is not what we built. The good news is that for catching data theft, the envelope is often enough: a big upload to an unfamiliar machine at a suspicious time is a strong signal on its own.

Timestomping (the first thing we detect)

"Timestomping" is an anti-forensic trick. An attacker drops a malicious file on a machine and then rewrites its timestamps so it looks old, like it has been a normal part of Windows for years. It is called timestomping because the attacker stomps on the real timestamps. We catch it using the two-clocks idea and the logbook above.

Exfiltration (the second thing we detect)

"Exfiltration" just means stealing data by sending it off the machine. The classic pattern is: collect or create a file, then upload it somewhere. On a timeline that looks like a file appearing or changing on disk, followed shortly by a large transfer leaving the machine. CTRE looks for exactly that pairing.

"Offline and read-only"

CTRE never touches the live machine and never changes the evidence. It opens the collected files for reading only. This is both safer (no risk to the original evidence) and more trustworthy (an investigator can show the tool could not have altered anything).

What CTRE produces

After reading the three artifacts, CTRE gives you:

- a list of files whose timestamps look faked, each with a reason,
- a list of transfers that line up with disk changes, and

- one **timeline** that merges every disk and network event in time order,

shown in a web page and downloadable as a report.

The project as one picture

Think of CTRE as an assembly line (this is Figure 1 in the paper):

- Three inputs go in on the left: the MFT, the change journal, and the pcap.
- Each goes through a **parser**, a piece of code that understands that file's format and pulls out the useful pieces.
- The parsed data feeds two **analysis** steps: one looks for faked timestamps, the other looks for exfiltration.
- The results come out as one **timeline and report**.

The code mirrors this exactly. Each stage is one file in the `ctre/` folder:

- `ctre/mft.py` and `ctre/usnjrnl.py`: the disk parsers.
- `ctre/pcap_meta.py`: the network parser.
- `ctre/timestomp.py`: the timestamp-tampering detector.
- `ctre/correlate.py`: the exfiltration matcher and the timeline builder.
- `ctre/report.py`: the report writer.
- `ctre/service.py`: the manager that runs all the stages in order.
- `ctre/web.py` and `ctre/web/`: the web app.

Keep this map in mind: each presenter owns a few of these files.

Part 1 - Vivek: the problem and the big picture

The idea in plain words

Vivek's job is to make the audience care about the problem before anyone talks about code. The problem is that the evidence from an attack is scattered across different places, and the clue that solves the case is often the *relationship* between two pieces, not either piece alone.

Why it matters to the project

Everything else in the project exists to join those scattered pieces. If the audience understands this, Kshitij's and Moksh's parts land as "here is how we join piece A and piece B," instead of as disconnected technical details.

What to explain, step by step

1. **The scatter problem.** A file lands on disk (the file system knows), the

system logs the change (the journal knows), and data leaves (the capture knows). Three witnesses, three separate notebooks. 2. **Why that is hard today.** Investigators use a different tool for each notebook and reconcile them by hand. Links get missed, especially timing links like "this file was created ten seconds before this upload." 3. **What CTRE does about it.** It reads all three notebooks at once and answers two questions automatically: were any timestamps faked, and did data leave right after a file changed? 4. **The two ground rules.** Everything is offline and read-only (we study copies, we never change evidence), and we never decrypt traffic (we use only what is already visible, which is enough for the theft question). 5. **The assembly line.** Walk through Figure 1 slowly: three inputs, parsers, two analysers, one timeline out.

Where this lives in the project

Vivek is explaining the shape of the whole system, so the relevant file is `ctre/service.py`, the manager that runs the stages in order (its `run_analysis` function calls each parser, then each analyser, then assembles the results). You do not need to read code to the audience; just say "one coordinator runs each stage in turn, and that is `service.py`."

Script

"When a machine gets attacked, the evidence is scattered. The file system knows a new file appeared. The file system's own logbook knows exactly when. And the network recording knows that data was sent out. The catch is that each of these is normally read with a different tool, so the connection between them, like a file being created right before a big upload, is easy to miss. Our project reads all three together. On this diagram you can see the whole flow: the three pieces of evidence on the left go through parsers that understand their formats, then into two analysis steps, one for faked timestamps and one for data being stolen, and everything comes out as a single timeline. Two rules we never break: we only ever read copies of the evidence, and we never decrypt anyone's traffic."

Analogy

Three security cameras recorded the same break-in from different angles, but each tape is in a different room and on a different machine. CTRE is the editing desk that lines all three up on one screen with a shared clock.

If asked

- *Why not just use Wireshark or an existing forensics tool?* Those are great at one artifact each. The value here is the automatic join across artifacts and the shared timeline.
- *Why offline?* Because real incident response works on collected copies, and touching the live machine can destroy evidence.
- *Is this a real product?* It is a student project that demonstrates the approach on controlled test data, not a finished commercial tool.

Part 2 - Kshitij: disk forensics and catching fake timestamps

The idea in plain words

Kshitij owns the disk side: reading the Windows file system records and using them to catch timestamp tampering. This is the heart of the project's forensic logic, so take the most time here.

Background to give the audience (keep it simple)

- Remind them of the **two clocks**: SI (easy to fake) and FN (hard to fake), both stored in a file's MFT entry.
- Remind them of the **logbook** (the change journal) that independently records when each file was really created or changed.
- Define **timestomping** again in one line: faking a file's timestamps so it looks like an old, trusted file.

Why it matters to the project

This part turns raw Windows data structures into the project's first headline result: "this file's timestamps were faked." Without it, CTRE would only have network data and no disk insight.

What the code actually does

There are three files here.

`ctre/filetime.py` - making the clocks comparable. Windows stores times in an odd format called

FILETIME (a count of 100-nanosecond ticks since the year 1601). This small file converts those into normal dates so every other part of the project can compare times cleanly. It also has a helper that checks whether a timestamp has its fractional seconds set to exactly zero, which matters for one of the detection rules.

`ctre/mft.py` - reading the file records. Its `parse_mft` function reads the MFT one record at a time. For each record it:

- applies a built-in NTFS integrity fix-up (a check NTFS uses to detect records that were half-written), then
- pulls out both the SI and FN timestamp sets and the file name.

If a record is damaged or malformed, it is skipped rather than crashing the run, because evidence files cannot be trusted to be perfect.

`ctre/usnjrnl.py` - reading the logbook. Its `parse_usnjrnl` function reads the change journal. Two real-world details are worth mentioning:

- A real journal can be several gigabytes, most of it empty padding, so the code memory-maps the file and skips over the empty regions instead of loading it all into memory. A one-gigabyte sparse test file reads in about 4.5 seconds.
- Each journal entry carries a "reason" (create, rename, write, delete) and a time, which the detector relies on.

`ctre/timestomp.py` - the detection itself. Its `analyze` function compares the clocks and the logbook and raises findings. The four rules (this is Figure 2 in the paper) are:

- `SI_BEFORE_FN` (HIGH): the file's easy clock says it was created before its hard clock. A file cannot be created before its own name record, so the easy clock was pushed into the past.
- `SI_BEFORE_JRNL` (HIGH): the easy clock predates the logbook's record of the file being created. The logbook cannot mention a file before it exists.
- `ZERO_SUBSECOND` (MEDIUM): several of the easy-clock times have their fractional seconds set to exactly zero, which many timestomping tools produce.
- `NO_JRNL_FOR_CHANGE` (MEDIUM): the easy clock says the file was modified more recently than anything in the logbook for that file, a change the logbook never saw.

Each finding includes a severity and a sentence explaining it.

The clever part worth emphasizing (avoiding false alarms)

Two of these rules would wrongly accuse normal files without extra care, and fixing that is a genuine engineering highlight:

- The logbook is a **ring buffer**: it only keeps recent history and overwrites the oldest entries. So for an old file, the earliest surviving logbook entry is usually not its creation. The `SI_BEFORE_JRNL` rule therefore only fires when the logbook actually holds a *creation* entry for the file, not just an old edit.
- Windows **reuses MFT record numbers** after a file is deleted. That means logbook history from a long-gone file can look like it belongs to the current file in the same slot. The detector also checks a "sequence number" that tells the old and new occupants apart, so a deleted file's history is not blamed on its replacement.

Both of these have their own tests, so they cannot silently regress.

Script

"Windows keeps two timestamps for every file. One is easy to change, one is not. On top of that, the system keeps a logbook of when files really change. Our detector plays these against each other. If the easy clock claims a file is from 2019, but the hard clock and the logbook both say it actually appeared last

week, that gap is the tampering, and we flag it. This flowchart shows the exact checks we run, and each one comes with a severity and a plain-English reason. The part I am proudest of is that we handle the awkward real-world cases: the logbook fills up and forgets old history, and Windows recycles file slots, so we added guards so that normal old files are not wrongly accused."

Analogy

A painting (the file) has a date written on the front by the owner (easy to change) and a date stamped on the back by the gallery (hard to change), plus the gallery's intake register (the logbook). If the front says 1850 but the back and the register say the painting arrived last month, it was backdated.

If asked

- *Could innocent files get flagged?* The two HIGH rules have the guards above. The ZERO_SUBSECOND rule is softer: files unzipped from an archive or copied from an older file system can also have zeroed fractions, so we mark it MEDIUM and call it a lead, not proof.
- *What if the journal is enormous?* It is memory-mapped and the empty space is skipped, so size is not a problem.
- *Do you need the whole disk?* No, just the extracted MFT and journal files.

Part 3 - Moksh: network evidence and connecting it to the disk

The idea in plain words

Moksh owns the network side: reading the packet capture without decrypting it, and then connecting network activity to the disk activity Kshitij found. This is where the two halves of the project meet.

Background to give the audience

- A packet capture is a recording of network traffic.
- Most traffic is encrypted, so you can see the envelope (who, when, how big) but not the contents. We deliberately stay with the envelope.
- Even on encrypted connections, a couple of things are sent in plain text: the website name looked up in DNS, and the server name in the start of a secure connection. We can read those.

Why it matters to the project

This part produces the project's second headline result: "a large transfer left the machine right when a suspicious file changed on disk." It depends on the disk side (for the "when a file changed" part), which is why Moksh presents after Kshitij.

What the code actually does

Two files.

`ctre/pcap_meta.py` - reading the traffic. Its `parse_pcap` function reads the capture and builds a **flow** record for each connection. A flow summarizes one conversation: the two machines, the ports, how many packets and bytes went each way, and the first and last time it was seen. It also extracts two plain-text details without any decryption: DNS query names (the website names being looked up) and the server name offered at the start of a secure connection (called the SNI). It explicitly does no decryption anywhere.

`ctre/correlate.py` - joining the worlds and building the timeline. This has two jobs:

- `correlate_exfil` looks for a large outbound flow whose start time is within a chosen window (five minutes by default) of a disk change recorded in the logbook. When it finds one, it records a

correlation: "this transfer lines up with this file change." This is Figure 3 in the paper.
- `build_timeline` merges every event, disk and network, into one list sorted by time, so the story reads top to bottom.

Why keyless analysis is the right call (make this a feature, not a gap)

The original project brief included reading encryption keys from memory and decrypting the traffic. The team left that out on purpose, because the same code would decrypt anyone's private traffic, which is an eavesdropping tool. The point to make: we do not need it. The theft signal is the size, timing, and destination of a transfer, and all of those are visible without decryption. This shows a deliberate ethical and security decision.

Script

"We do not break encryption, and we do not need to. A recording of the traffic still tells us who the machine talked to, when, and how much data left, plus the website names from DNS and the server name in the secure handshake. Our tool takes every large outbound transfer and asks: did a file change on disk right around the same time? When the answer is yes, that is the classic shape of data being staged and then stolen, and we put it on the timeline right next to the disk events. So Kshitij's side tells us a file was tampered with, and my side can show that same file's data then left the machine."

Analogy

A warehouse has a stock log (the disk changes) and a loading-dock camera (the network capture). You cannot see inside the boxes being loaded (encryption), but if a box appears in the stock log and a truck of about the same size pulls out two minutes later, you have a strong story without opening the box.

If asked

- *How do you know a transfer is "outbound" and "large"?* The flow records byte counts in each direction, and we apply a size threshold that the user can change in the app.
- *Could slow, trickled-out theft be missed?* Yes. Data sent very slowly over a long time can stay under the size and time thresholds. That is a known limit.
- *What formats does it read?* The common capture format with standard Ethernet and IPv4. It tells the user to convert other formats rather than failing silently.

Part 4 - Aranck: the app, testing, safety, and results

The idea in plain words

Aranck makes the project real and trustworthy: the app people actually use, the proof that it works, the safety of running it, and the results on the test case. This is also the part that drives the live demo.

Why it matters to the project

A correct analysis is useless if nobody can run it and nobody can trust it. This part is what turns the engine into something presentable and believable.

What the code actually does

The app: `ctre/web.py` and `ctre/web/`. Running the tool with no arguments starts a small web server on your own machine and opens a page in the browser. The page (in `ctre/web/`) explains the project as you scroll: an overview with the pipeline diagram, numbered steps, a one-click sample case, the results, and a reference section with the rules and a glossary. After a run it prints a plain summary of what it found. Behind the page, `ctre/service.py` runs the exact same analysis as the command line, so the

app and the command line always agree.

The report: `ctre/report.py`. Produces a single self-contained HTML report and a JSON file. Every piece of text taken from the evidence is escaped before it is written, so a booby-trapped file name cannot inject anything into the report.

The safety of the local server (worth spelling out, because "a web server" can sound risky):

- It listens only on your own machine (the loopback address), not on the network.
- It checks the address it was asked for, so another site cannot reach it by name.
- It requires a special header on every request, which stops other websites open in your browser from quietly driving it.
- It sends a strict policy that limits what the page can load, and it deletes any uploaded evidence when it stops.
- Text from the evidence is always placed as plain text, never as code.

The proof: `ctre/gen_testdata.py` and the `tests/` folder. Because the team did not want to rely on real evidence, there is a generator that writes fake artifacts in the exact formats the parsers read, so the generator and the parsers check each other. It plants one timestomped file and one timed transfer, so there is a known correct answer. There are **20 tests** covering the time math, each parser, the four detection rules, the correlation, the report escaping, and a full upload-run-download cycle against the live server. All pass. On top of that, the network parser was cross-checked against scapy, a separate and widely used packet library, and the packet count, DNS name, server name, and byte totals all matched.

The results to report (the sample case)

- Parsed 2 MFT entries, 4 logbook records, 122 packets forming 2 flows.
- Flagged the planted file `svchost.exe` with three rules: two HIGH (`SI_BEFORE_FN`, `SI_BEFORE_JRNL`) and one MEDIUM (`ZERO_SUBSECOND`).
- Did **not** flag the normal file (no false alarm).
- Linked the roughly 160 KB outbound transfer to the disk change for that file.
- The timeline tells the story: `svchost.exe` claims a 2019 creation in the MFT, the logbook records its real creation in the present, then a DNS lookup, a secure handshake, and the transfer to the outside host.

Script

"You run one command and this page opens. It is built to explain itself, so even a new user can follow it. I will click Load sample case, then Run analysis. Here is what it found: it caught the file we planted, flagged it with three separate rules, and left the normal file alone, which shows it is not just flagging everything. It also tied the data leaving the machine to the moment that file changed. The timeline reads like a story. And none of this is on trust: there are twenty automated tests that all pass, and we checked our network reader against a separate, well-known library to be sure the numbers are real. The app itself is safe to run: the server only listens on your own machine and has several guards so no website can reach it."

Analogy

If Kshitij and Moksh are the detectives, Aranck is the courtroom: the same findings, but now presented so anyone can follow them, with receipts (the tests) that prove the method works on a case where we already know the answer.

If asked

- *Is running a local web server dangerous?* No. It is only reachable from your own machine and has the guards listed above.

- *How confident are you in the results?* The pipeline is proven end to end on controlled data where we know the right answer. It has not yet been measured on real intrusions; that is the honest next step.
- *What does "all tests pass" really prove?* That each part behaves correctly on known inputs and that the whole chain works from upload to downloaded report. It is strong evidence of correctness, not a formal security proof.

Live demo runbook (one person drives, the rest answer)

1. Run `python -m ctre` to open the web page.
2. Click **Load sample case**, then **Run analysis**.
3. Read the **What the engine found** summary out loud.
4. Scroll to the **findings** and point out the two HIGH rules on `svchost.exe`.
5. Scroll to the **correlation** and then the **timeline**.
6. Toggle the timeline source filters to show disk versus network events.
7. Click **Download HTML report** to show the shareable output.

If anything fails live, fall back to the screenshots and the report PDF.

Who owns which files (quick appendix)

- **Vivek**: the big picture; coordinator `ctre/service.py`, command line `ctre/cli.py`.
- **Kshitij**: `ctre/filetime.py`, `ctre/mft.py`, `ctre/usnjrnl.py`, `ctre/timestomp.py`.
- **Moksh**: `ctre/pcap_meta.py`, `ctre/correlate.py`.
- **Aranck**: `ctre/web.py`, `ctre/web/`, `ctre/report.py`, `ctre/gen_testdata.py`, `tests/`.

Glossary (plain language)

- **DFIR**: digital forensics and incident response; investigating a computer attack from collected evidence.
- **NTFS**: the file system Windows uses to organize files on a disk.
- **MFT**: NTFS's master index; one entry per file, holding its timestamps.
- **SI / FN**: a file's two timestamp sets; SI is easy to fake, FN is hard.
- **Change journal (`$UsnJrnl`)**: NTFS's append-only logbook of file changes.
- **Timestamp / FILETIME**: when a file was created or changed; Windows stores it in a special counting format that we convert to normal dates.
- **Timestomping**: faking a file's timestamps to make it look old and trusted.
- **Packet / pcap**: a chunk of network data; a pcap is a saved recording of them.
- **Flow**: one network conversation, summarized by who, ports, timing, and size.
- **DNS**: the system that turns a website name into an address; the names are visible even with encryption.
- **SNI**: the server name sent in the clear at the start of a secure connection.
- **Encryption**: scrambling data so only the two ends can read it.
- **Exfiltration**: stealing data by sending it off the machine.
- **Offline / read-only**: working on copies without changing the evidence.

Commands

- `python -m ctre` - open the web app.
- `python -m ctre gen sample_artifacts` - write the sample artifacts.
- `python -m ctre analyze --mft ... --usnjrnl ... --pcap ... --report out.html` - run from the command line.
- `python -m pytest -q` - run the 20 tests.